Department Of Software Engineering

SE-113 (Introduction to Software Engineering)

Software Testing

Software Testing

Help to reduce the risk of problem occurring.

Error, Fault and Failure

- ✓ A programmer makes an error (mistake).
- ✓ Error results in a defect (fault, bug).
- ✓ A failure occurs when a faulty piece of code is executed.

Validation & Verification

- ✓ Validation: Are we building the right software?
- ✓ Verification: Are we building the product right?
- ✓ The aim of verification is to check that the software must be stated functional and non-functional requirements.
- ✓ The aim of validation is to ensure that the software meets the customers specification.

Testing vs Debugging

- ✓ **Debugging: The process of finding, analyzing and removing the causes of failure.**
- ✓ **Testing is a quality assurance activities.**

Test Types

- ✓ Functional Testing
- ✓ Non-functional Testing

- ✓ Functional testing refers to tests that verify a specific action on function of the code. These are usually found in the code requirements documentation, although some development methodologies work from use cases or user stories.
- ✓ Non – functional testing refers to aspects of the software that may not be related to a specific function or user action, such as scalability or scalability or security. Non – functional testing tends to answer such question as
  'how many people can log in at once' or 'how easy is it to hack this software'.

Test Levels

- ✓ Unit/Component testing.
- ✓ Integration testing.
- ✓ System testing.
- ✓ Acceptance testing.
  - ✓ Alpha and Beta (field) testing.

Testing Techniques

- ✓ **White-Box Testing**
  - ✓ Glass box testing.
  - ✓ Structural testing
  - ✓ Guarantee that all development paths within a module have been exercised at least once.
  - ✓ Exercise all logical decisions on their true and false sides.
  - ✓ Execute all loops at their boundaries and within their operations bound.
  - ✓ Exercise internal data structures to ensure their validity.

Testing vs Debugging

- ✓ **Debugging: The process of finding, analyzing and removing the causes of failure.**
- ✓ **Testing is a quality assurance activities.**

Test Types

- Functional Testing
- Non-functional testing

Functional Vs Non-functional testing

Functional testing

- Functional testing refers to tests that verify a specific action or function of the code.
- These are usually found in the code requirements documentation, although some development methodologies work from use cases or user stories.
- Functional tests tend to answer the question of "**can the user do this**" or "**does this particular feature work**".

Non-functional testing

- Non-functional testing refers to aspects of the software that may not be related to a specific function or user action, such as scalability or security.
- Non-functional testing tends to answer such questions as "**how many people can log in at once**", or "**how easy is it to hack this software**".

Black Box Testing

- Specification Based

White Box Testing

- Structured Based Testing

Gray Box Testing

- Black Box testing + White Box Testing

Test Levels

- Unit / Component Testing
- Integration Testing
- System Testing
- Acceptance Testing
  - Alpha and Beta (field) testing

Testing Techniques

White-Box Testing

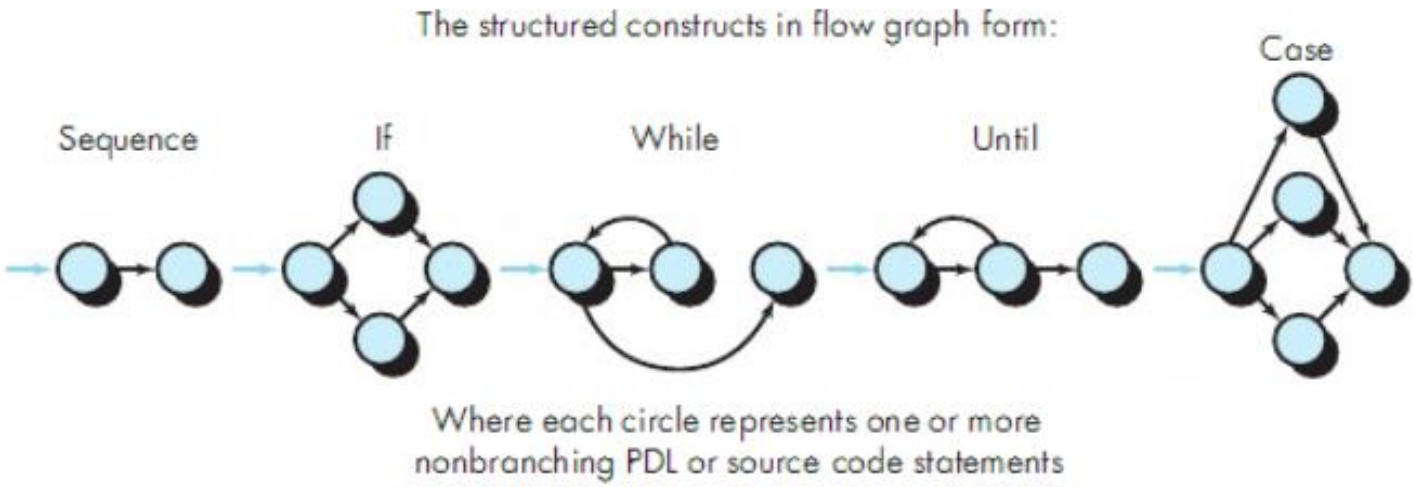
- Glass-box testing
- Structural testing

White-Box Testing

- Guarantee that all independent paths within a module have been exercised at least once
- Exercise all logical decisions on their true and false sides
- Execute al loops at their boundaries and within their operations bounds
- Exercise internal data structures to ensure their validity

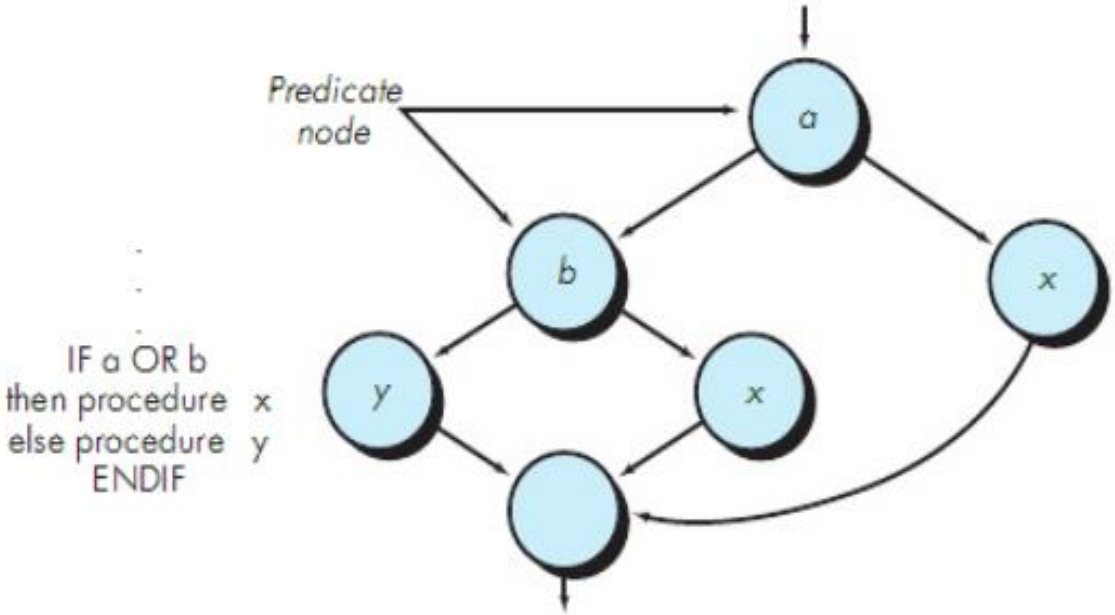
Basic Path Testing

- White-box testing techniques
- Flow graph (or Program Graph) Notation
  - A simple notation for the representation of control flow



Basic Path Testing

- A node containing a condition is called a predicate node.



How many paths we need to test?

Cyclomatic Complexity

- Is a software metric that provides a quantitative measure of the logical complexity of a program.
- It defines the number of independent paths in the basic set of a program.

Cyclomatic Complexity

- Complexity is computed in one of three ways:
  1. The number of regions of the flow graph corresponds to the cyclomatic complexity
  2. Cyclomatic complexity V(G) for a flow graph G is defined as

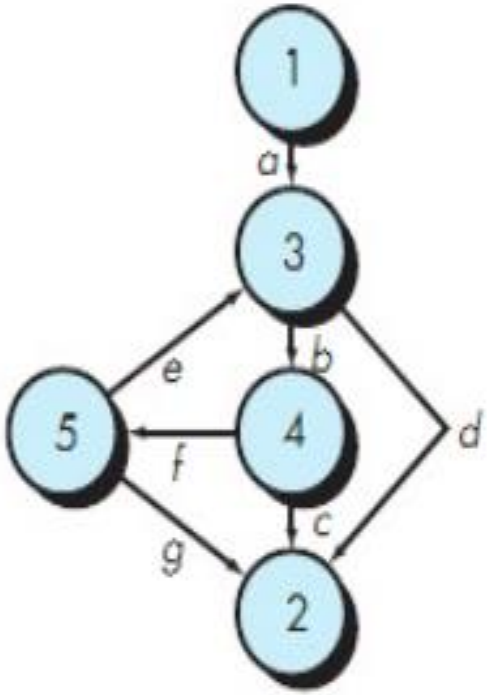
     V(G) = E – N + 2, where **E** is the number of flow graph edges and **N** is the number of flow graph nodes.
  3. Cyclomatic complexity V(G) for a flow graph G is defined as

     V(G) = P + 1, where **P** is the number of predicate nodes.

Graph Matrices

- Is a data structure that assists in basic path testing.
- It is a square matrix whose size is equal to the number of nodes on the flow graph

Graph Matrices



Flow graph

Node \ Connected to node	1	2	3	4	5
1			a		
2					
3		d		b	
4		c			f
5		g	e		

Graph matrix

Connection matrix

Node \ Connected to node	1	2	3	4	5	Connections
1			1			1 – 1 = 0
2						
3		1		1		2 – 1 = 1
4		1			1	2 – 1 = 1
5		1	1			2 – 1 = 1

Graph matrix

3 + 1 = 4 ← Cyclomatic complexity

Black Box testing

- Black box testing is testing that ignores the internal mechanism of a system or component and focuses only on the outputs generated in response to selected inputs and execution conditions.

Boundary Value Analysis (BVA)

- A test design that focus on finding range related errors
- **Test cases for input box accepting numbers between 1 and 1000 using Boundary value analysis:**

  -Test cases with test data exactly as the input boundaries of input domain i.e. values 1 and 1000 in our case.

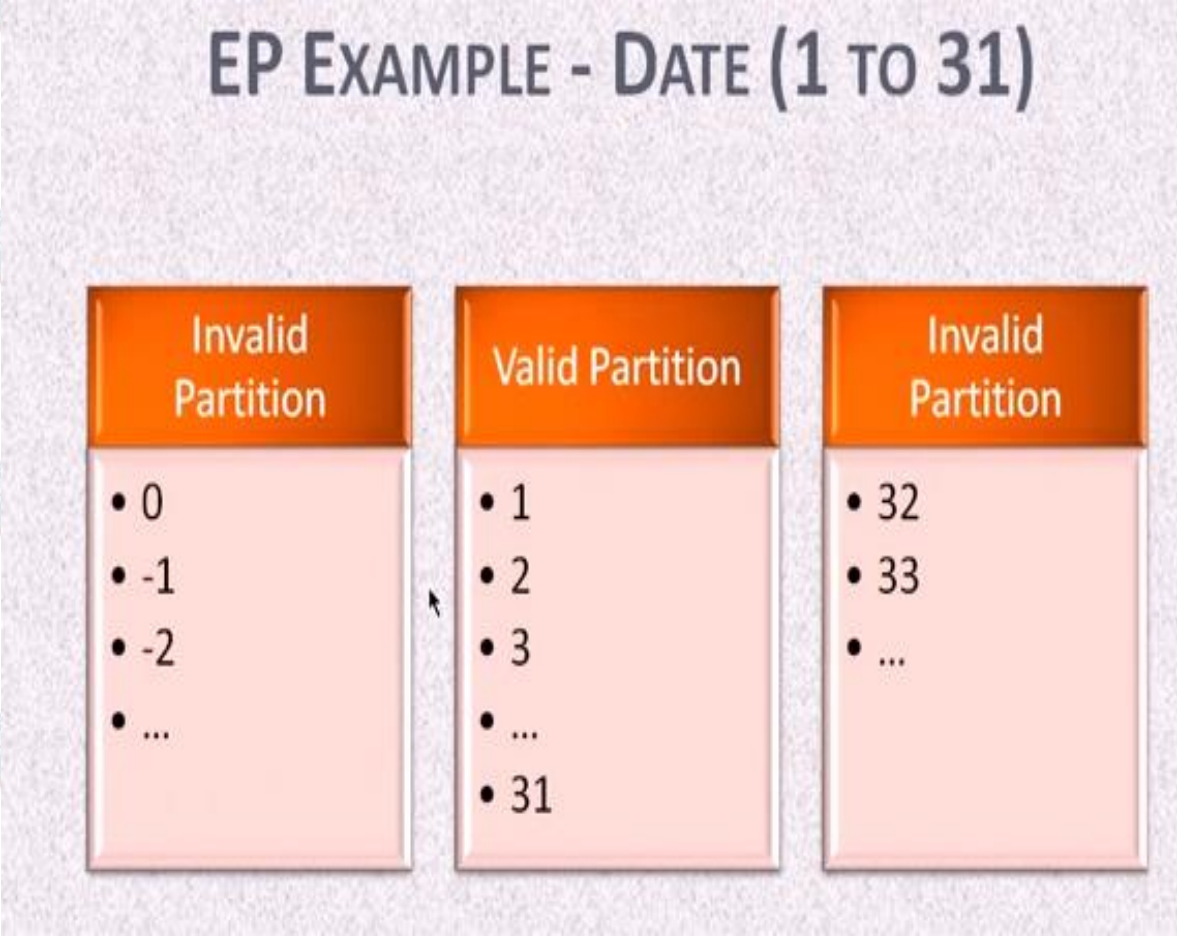
  -Test data with values just below the extreme edges of input domains i.e. values 0 and 999.

  - Test data with values just above the extreme edges of input domain i.e. values 2 and 1001.

Equivalence Partitioning (EP) or Equivalence Class Partitioning (ECP)

- Equivalence partitioning (also called Equivalence Class Partitioning or ECP) is a software testing technique that divides the input data of a software unit into partitions of equivalent data from which test cases can be derived. In principle, test cases are designed to cover each partition at least once. This technique tries to define test cases that uncover classes of errors, thereby reducing the total number of test cases that must be developed. An advantage of this approach is reduction in the time required for testing a software due to lesser number of test cases.

Equivalence Partitioning (EP) or Equivalence Class Partitioning (ECP)



Equivalence Partitioning (EP) or Equivalence Class Partitioning (ECP)

EP Example - Username (6 to 10 characters)

Invalid Partition	Valid Partition	Invalid Partition
• 0 character • 1 character • 2 characters • 3 characters • 4 characters • 5 characters	• 6 characters • 7 characters • 8 characters • 9 characters • 10 characters	• 11 characters • 12 characters • ...

Equivalence Partitioning (EP) or Equivalence Class Partitioning (ECP)

EP Example - Age (18 to 80 years except 60 to 65 years)

Invalid Partition	Valid Partition	Invalid Partition	Valid Partition	Invalid Partition
• ... • 0 year • 1 year • 17 years	• 18 years • ... • 59 years	• 60 years • ... • 65 years	• 66 years • ... • 80 years	• 81 years • 82 years • ...

Boundary Value Analysis (BVA)

BVA Example - Date (1 to 31)

Invalid Partition – Valid Partition Lower Boundary		Invalid Partition – Valid Partition Upper Boundary	
Boundary value just below the boundary	Boundary value just above the boundary	Boundary value just below the boundary	Boundary value just above the boundary
0	1	31	32

Boundary Value Analysis (BVA)

BVA Example - UserName (6 to 10 characters)

Invalid Partition – Valid Partition Lower Boundary		Invalid Partition – Valid Partition Upper Boundary	
BV just below the boundary	BV just above the boundary	BV just below the boundary	BV just above the boundary
5 characters	6 characters	10 characters	11 characters

Boundary Value Analysis (BVA)

BVA Example – Floating Point Values (0.2 to 0.8 – one decimal place)

Invalid Partition – Valid Partition Lower Boundary		Invalid Partition – Valid Partition Upper Boundary	
BV just below the boundary	BV just above the boundary	BV just below the boundary	BV just above the boundary
0.1	0.2	0.8	0.9

Decision Table

- Document the complicated logic
- Help to test all combinations of conditions
- Contains condition and action

Decision Table

DECISION TABLE TESTING – EMAIL EXAMPLE

Email	
Password	

Email	F			
Password	F			
Expected result	Error: Please enter email.			

Decision Table

DECISION TABLE TESTING – EMAIL EXAMPLE

Email	SoftwareAndTestingTraining
Password	

Email	F	T		
Password	F	F		
Expected result	Error: Please enter email.	Error: Please enter password.		

Decision Table

DECISION TABLE TESTING – EMAIL EXAMPLE

Email	
Password	*****

Email	F	T	F	
Password	F	F	T	
Expected result	Error: Please enter email.	Error: Please enter password.	Error: Please enter email.	

Decision Table

DECISION TABLE TESTING – EMAIL EXAMPLE

Email	SoftwareAndTestingTraining
Password	*****

Email	F	T	F	T
Password	F	F	T	T
Expected result	Error: Please enter email.	Error: Please enter password.	Error: Please enter email.	Login processed.

Decision Table

DECISION TABLE TESTING – EMAIL EXAMPLE

Conditions	Email	Blank	Blank	Blank	Invalid	Invalid	Invalid	Valid	Valid	Valid
	Password	Blank	Invalid	Valid	Blank	Invalid	Valid	Blank	Invalid	Valid
Actions	Expected result	Error: Please enter email.	Error: Please enter email.	Error: Please enter email.	Error: Please enter a valid email.	Error: Login failed.	Error: Please enter a valid email.	Error: Please enter password.	Error: Login failed.	-
	Show Page	Login page	Login page	Login page	Login page	Login page	Login page	Login page	Login page	Home page

Decision Table

DECISION TABLE TESTING – LIBRARY EXAMPLE

Conditions	User registered?	F	F	F	F	T	T	T	T
	No outstanding fees?	F	F	T	T	F	F	T	T
	Under borrow limit?	F	T	F	T	F	T	F	T
Actions	Borrow book?	No	No	No	No	No	No	No	Yes